Hands-on No. : 1

Topic : Strings

Date : 12-12-25

1.Write a Java method that removes all duplicate characters from a given string while preserving the order of first appearance.

Constraints:

- Ignore case for duplication comparison (optional based on requirement)
- Do not use Set<String> built-in removal methods
- Use your own logic + frequency/boolean array

Sample Input:

corporate assignment

Sample Output:

corpte asingm

2.Write a custom method myTrim(String s) that removes leading and trailing spaces from a string without using built-in trim().

Constraints:

- Do not use: trim(), strip(), regex
- Preserve internal spaces exactly as they are

Sample Input:

" Welcome to Java "

Sample Output:

"Welcome to Java"

3. Validate corporate email formats

Write a method that validates corporate email addresses using a single regular expression. The validator must accept common corporate formats such as name.surname@techm.com and firstname_lastname@infosys.co.in and reject invalid formats such as emails that:

- start with a number (i.e., numbers at the beginning of the local-part are not allowed),
- contain consecutive dots anywhere,

- missing a domain or top-level domain (TLD) (e.g., missing the @example.com part),
- local-part or domain labels beginning/ending with invalid punctuation,
- have consecutive punctuation characters in the local-part (like .., ._, -., etc.)

Requirements / constraints

- The local-part:
  - Must start with a letter (A–Z, case-insensitive).
  - May contain letters, digits, dot (.), underscore (_) and hyphen (-) after the first letter.
  - Must not contain consecutive punctuation (e.g., .., ._, -.).
  - Must not start or end with ./_/-.
- The domain:
  - Must contain at least one dot (so a TLD exists).
  - Domain labels (pieces between dots) must start and end with a letter or digit; hyphens are allowed inside a label but not at the start or end of a label.
  - Support multi-part TLDs like .co.in.

4. Write a method to encrypt a message using a simplified corporate cipher. The encryption rules must transform the input string character-by-character as follows:

   Encryption Rules

   1. Alphabet characters (A–Z, a–z)
      - Shift each alphabet by +3 positions (Caesar shift).
      - Preserve the original case (uppercase stays uppercase; lowercase stays lowercase).
      - Wrapping must occur:
        - X → A
        - Y → B
        - Z → C
        - x → a
        - y → b
        - z → c

2. Digits (0–9)
   - Keep digits unchanged.
3. Spaces
   - Replace each space with an underscore (_).
4. All other characters
   - Copy them as-is (unless you want to restrict them — optional).

5. You are given a raw application log entry containing multiple key–value pairs. Some fields contain sensitive information and must be sanitized before storing or displaying the log.

   Write a method to sanitize the log entry according to the rules defined below.

   Input Example

   User=Richard; Password=ricadmin@123; IP=192.168.1.1; Status=SUCCESS

   Sanitization Rules

   1. Password Masking

   - Replace the value of the Password field with masked characters.
   - Masking format: Password=********
     - The number of asterisks does **not** need to match the actual password length unless specified (you may choose fixed or dynamic masking).

   2. IP Redaction

   - Replace any IPv4 address with a fully redacted pattern: xxx.xxx.xxx.xxx
   - Only digits and dots must be replaced; the rest of the log must remain unchanged.

   3. Other Fields

   - Keep all non-sensitive fields as-is (e.g., User, Status, Timestamp fields, etc.)

   Expected Output

   User=Richard; Password=**********; IP=xxx.xxx.xxx.xxx; Status=SUCCESS

Requirements

- You must use **string manipulation or regex**, not external security libraries.
- Logic must work even if fields appear in different order.
- Password may include letters, digits, and special characters.
- IP may be any valid IPv4 address (0–255 range need not be validated; treat it as any sequence of digits separated by dots).